Experiment No.: 9

1. Aim: - Write a program for character recognition using RNN and compare with CNN.

2. Objective: -

- To implement a character recognition system using a Recurrent Neural Network (RNN).
- To compare the performance of RNN with a Convolutional Neural Network (CNN) for character recognition tasks.

3. Outcome: -

- Students will understand how deep learning models can be applied for character recognition.
- Students will learn the working of RNN and CNN architectures for image-based classification.
- Students will analyze and compare the performance of RNN and CNN models.

Software Required: Python

4. Theory:

The program is to implement a dog detection system using the YOLO (You Only Look Once) object detection algorithm in Python. This system will be capable of identifying and locating dogs within images. It demonstrates the capabilities of deep learning techniques in computer vision applications. Character recognition is a process of identifying and classifying characters from images or scanned documents. It is widely used in applications such as Optical Character Recognition (OCR), handwritten digit recognition, and document digitization.

Recurrent Neural Networks (RNNs) are deep learning models designed for sequential data. RNNs contain loops that allow information to persist, making them useful for tasks involving sequence prediction. In character recognition, RNNs can process sequences of pixels or extracted features.

Convolutional Neural Networks (CNNs) are widely used for image processing tasks. CNNs use convolutional layers to automatically extract spatial features from images. They are highly effective in image classification tasks such as digit recognition.

Comparison between RNN and CNN

- **CNN:** Best suited for image recognition because it extracts spatial features using convolution filters.
- **RNN:** Better suited for sequential data and temporal dependencies.
- In character recognition, CNN generally provides better accuracy for image-based inputs, while RNN can be useful when characters appear in sequences.

5. Design/ Algorithm

1. Import necessary libraries (NumPy, TensorFlow/Keras, Matplotlib).
2. Load the dataset (e.g., MNIST handwritten digits dataset).
3. Preprocess and normalize the image data.
4. Split the dataset into training and testing sets.
5. Build an RNN model for character recognition.
6. Train the RNN model on the training dataset.
7. Build a CNN model for the same dataset.
8. Train the CNN model.
9. Evaluate both models using accuracy and loss metrics.
10. Compare the performance of RNN and CNN.

6. **Step-by-Step Procedure**

1. Install required Python libraries such as TensorFlow, Keras, NumPy, and Matplotlib.
2. Import the necessary libraries in Python.
3. Load the MNIST dataset for handwritten character recognition.
4. Normalize the dataset for better model performance.
5. Reshape the dataset according to the RNN and CNN input requirements.
6. Create and train the RNN model.
7. Create and train the CNN model.
8. Evaluate both models using test data.
9. Plot the training and testing accuracy for comparison.
10. Analyze the results and identify the better performing model.

6. Conclusion: -

In this experiment, character recognition was implemented using both RNN and CNN models. The models were trained on a dataset of handwritten characters and evaluated for accuracy. The comparison showed that CNN performs better for image-based recognition tasks due to its ability to capture spatial features effectively. RNN models can also recognize characters but are generally more suitable for sequential data. This experiment demonstrates the advantages of deep learning techniques for pattern recognition tasks.

7. Viva Question

1. What is object detection in computer vision?

2. What is YOLO and how does it work?

3. What does YOLO stand for?

4. What are the advantages of YOLO compared to traditional object detection methods?

5. What is the role of the **COCO dataset** in YOLO?

6. What is a **bounding box** in object detection?

7. What is **confidence score** in object detection?

8. References

1. M. Mitchell, Machine Learning, New York, NY, USA: McGraw-Hill, 1997.
2. K. P. Murphy, Machine Learning: A Probabilistic Perspective, Cambridge, MA, USA: MIT Press, 2012.
3. G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning with Applications in R, New York, NY, USA: Springer, 2013.
4. TensorFlow Deep Learning Tutorials – https://www.tensorflow.org/tutorials
5. Keras Documentation – https://keras.io